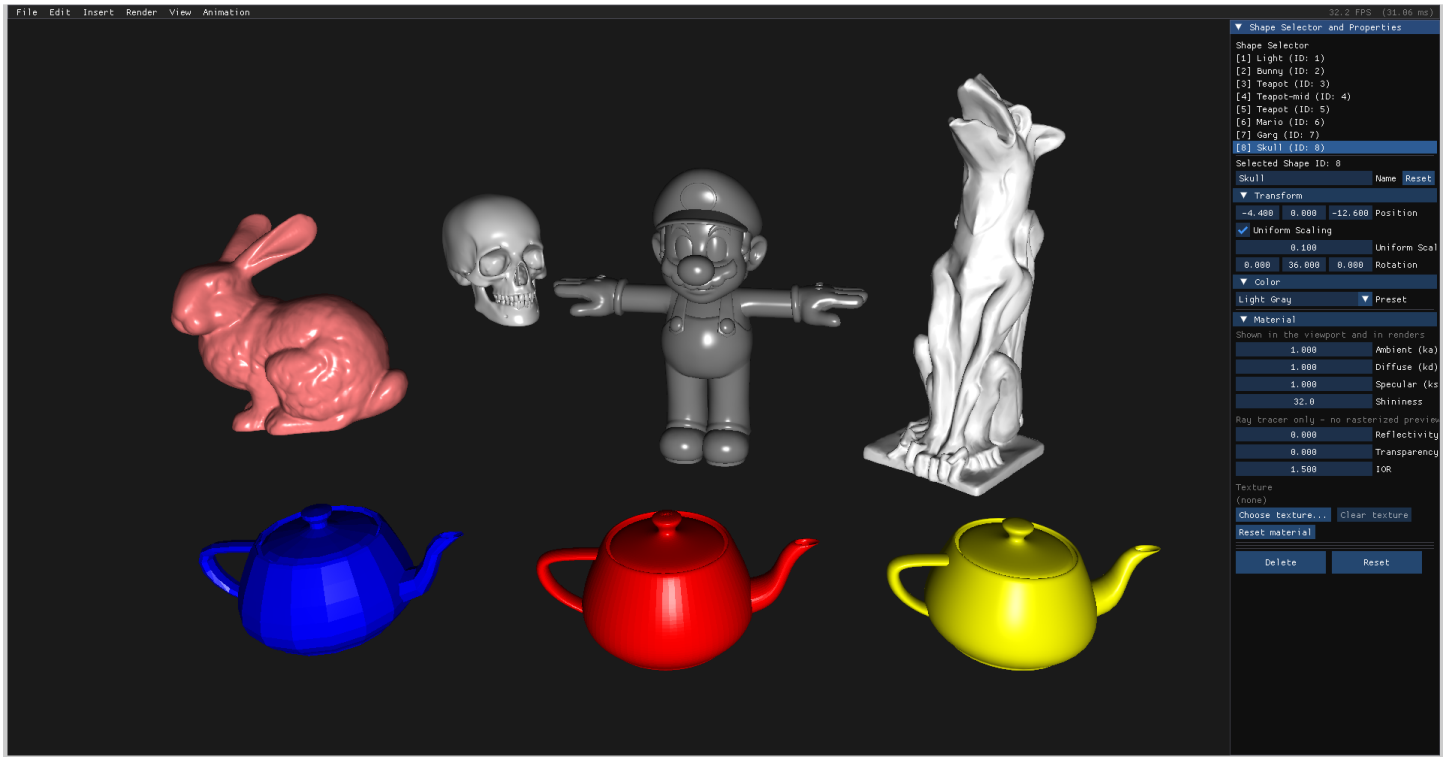
CALIFORNIA STATE UNIVERSITY FULLERTON

Assignment 3

CPSC 484 – Principles of Computer Graphics
Assignment 3: OBJ Importer and Camera Control
Fall 2026
Professor: Dave Garcia-Gomez
Version: 2026-10-07



1 Getting Started

In this assignment, you will extend the 3D Model Scene Editor developed in the previous assignments by implementing two related pieces of functionality: an interactive camera system and an OBJ file importer. The camera system will allow you to navigate a scene by moving the viewpoint around a target, while the OBJ importer will allow the editor to load mesh data that was created outside of the editor.

These two features address an important limitation of the editor as it currently exists. In Assignment 2, the camera remained fixed while you developed the modeling and transformation systems. In this assignment, you will implement the camera controls that allow you to inspect a scene from different viewpoints. You will also extend the editor beyond procedurally generated shapes by reading mesh data from OBJ files.

CALIFORNIA STATE UNIVERSITY FULLERTON

Assignment 3

The primary objectives of this assignment are to:

- Implement the orbit, pan, and zoom operations for the editor's camera;
- Use spherical coordinates to maintain the camera position relative to a target point;
- Parse the required subset of the Wavefront OBJ file format;
- Construct the imported mesh from the vertex, normal, and face information contained in the OBJ file; and
- Integrate both features into the existing editor without changing the portions of the application that have already been provided.

Before beginning the implementation, review the existing editor source code and identify where the camera state, mouse input, file-import dialog, and mesh data structures are defined. Several of the supporting components are already implemented for you. Your responsibility is to complete the specific methods identified in Sections 3 and 4 rather than redesigning the surrounding systems.

2 Building the Editor and Using Solution Libraries

As you work through the assignments, you will sometimes encounter functionality that was introduced in an earlier assignment but that you have not yet completed. Rather than requiring you to stop working until that functionality is finished, the editor supports **solution libraries**.

A solution library is a precompiled version of one part of the editor. You can add a supplied library to your project and tell the build system to use that completed implementation instead of your unfinished implementation.

This allows you to continue working on the current assignment while still using functionality from a previous assignment. This section is similar, but not exactly, the same as Assignment2 and is included just for your reference.

2.1 Building the Editor

From the root directory of the editor project, extract the project if necessary and build it with the following commands:

```
tar xvzf editor.tar.gz
cd editor
make
```

If the build succeeds, an executable named editor will be created. Run the executable from the project directory:

```
./editor
```

You should run the editor from the project directory because the application loads resources such as shaders using paths relative to the current working directory.

2.2 Cleaning and Rebuilding

When you make changes to your source code, make will normally rebuild only the files that need to be recompiled.

If you want to remove the compiled object files and rebuild the editor from scratch, use:

```
make clean
```

Then rebuild the editor:

```
make
```

You can also combine the commands:

```
make clean && make
```

You should use make clean whenever you change which implementation of a topic is being used. This is especially important when switching between your implementation and a supplied solution library.

2.3 Using a Solution Library

Some functionality is implemented in later and earlier assignments. To allow you to use that functionality while working on your current assignment, solution libraries may be provided for your development environment.

Solution libraries are precompiled `.a` files. **Because compiled libraries are specific to the operating system, CPU architecture, and compiler/toolchain, you must use the library provided for your environment.** Do not copy a library from a different environment.

For example, if you want to use the completed camera implementation, you may be provided with:

```
libcamera.a
```

Step 1: Obtain the Library for Your Environment

Your instructor will provide the appropriate solution library separately based on your development environment. For example, the library provided for Linux may be different from the library provided for Windows or macOS.

Make sure you have the correct `.a` file for your environment before continuing.

Step 2: Copy the Library into `lib/solution/`

From the root directory of your editor project, copy the provided `.a` file into the `lib/solution/` directory.

Your directory should look similar to:

```
editor/
├──── lib/
│     └──── solution/
│           └──── libcamera.a
├──── src/
├──── Makefile
└──── ...
```

On Linux or macOS, you can copy the file using:

```
cp /path/to/libcamera.a lib/solution/
```

For example, if `libcamera.a` is in your Downloads directory:

```
cp ~/Downloads/libcamera.a lib/solution/
```

On Windows using MSYS2/MinGW, you can use:

```
cp /path/to/libcamera.a lib/solution/
```

or copy the file using Windows File Explorer and place it directly in:

```
editor\lib\solution\
```

Do not add the `.a` file manually to the `Makefile` or to `LIBS`. The build system will automatically locate and link the appropriate solution library when you use the corresponding build option.

Step 3: Build Using the Solution Library

Solution libraries are selected using a Make variable corresponding to the functionality they provide.

For example, to use the camera solution:

```
make clean
make CAMERA=solution
```

Then run the editor:

```
./editor
```

The CAMERA=solution option tells the build system to use the provided libcamera.a implementation instead of the camera implementation in your source tree.

For example, the following command:

```
make TRANSFORMS=solution
```

would use the provided transformation solution library instead.

The available solution libraries and their corresponding build options will be identified in the assignment instructions.

Important: Solution Libraries Are Environment-Specific

The source code for the assignment is shared across development environments, but the compiled `.a` libraries are not necessarily interchangeable.

For example:

```
Linux library       → Use with Linux
Windows library     → Use with Windows/MSYS2
macOS ARM library   → Use with Apple Silicon macOS
macOS Intel library → Use with Intel macOS
```

Always use the library supplied for your operating system, CPU architecture, and development environment. If you use an incompatible library, the linker may report errors or the resulting program may not run.

You do **not** need to modify the source code or Makefile to use a solution library. Copy the appropriate `.a` file into `lib/solution/` and use the corresponding `TOPIC=solution` build option.

Available Solution Libraries

The editor supports several optional solution libraries. These libraries allow you to temporarily use an instructor-provided implementation of a topic that you have not completed yet, so that you can continue working on later functionality.

For example, camera controls are part of a later assignment. While working on Assignment 2, you may use the provided camera solution library so that you can move around the scene while testing your transformations and geometry.

The editor currently recognizes the following solution libraries:

Functionality	Build Option	Library File
Model Transformations	TRANSFORMS=solution	libtransforms.a
Procedural Geometry and Normals	GEOMETRY=solution	libgeometry.a
OBJ Parsing	OBJPARSER=solution	libobjparser.a
Orbit, Pan, and Zoom Camera Controls	CAMERA=solution	libcamera.a
VR Camera / View Composition	VRCAMERA=solution	libvrcamera.a
Curves and Surfaces	CURVES=solution	libcurves.a

Functionality	Build Option	Library File
Skeletal Skinning	SKINNING=solution	libskinning.a
Keyframe Animation	ANIMATION=solution	libanimation.a
Texture Coordinates and Sampling	TEXTURES=solution	libtextures.a
Phong Lighting	LIGHTING=solution	liblighting.a
Ray-Tracer Shading	RAYTRACER=solution	libraytracer.a
Ray Intersections and BVH	BVH=solution	libbvh.a
Particle Integration and Forces	PARTICLES=solution	libparticles.a

Important: You should only use solution libraries that have been provided by your instructor. Some of the libraries listed above correspond to future assignments and may not be distributed until later in the semester.

2.4 Building the Solution Binary for Assignment 3

Assignment 3's solution file should include your code from Assignment 2 along with the solution files for Assignment 3. To build a solution binary from scratch, use the following libraries:

```
lib/solution/libtransforms.a
lib/solution/libgeometry.a
lib/solution/libobjparser.a
lib/solution/libcamera.a
```

If you want to build a solution binary with your existing code, all you will need is the following libraries:

```
lib/solution/libobjparser.a
lib/solution/libcamera.a
```

Then clean your previous build:

```
make clean
```

Build the editor using the camera and OBJ parser solutions:

```
make CAMERA=solution OBJPARSER=solution
```

To build the binary with Assignment 2 solutions as well:

```
make GEOMETRY=solution TRANSFORMS=solution CAMERA=solution
OBJPARSER=solution
```

Run the editor:

```
./editor
```

The editor will now use the completed camera implementation and your OBJ loader and parser while the rest of the application continues to use your own source code, or your other solutions files specified.

The camera controls are:

Mouse Action	**Camera Control**
Left-click and drag	Pan the camera
Right-click and drag	Rotate the camera
Mouse wheel	Zoom the camera

This allows you to work on Assignment 2 while using camera functionality that you will implement yourself in Assignment 3.

2.5 Returning to Your Own Implementation

Solution libraries are optional. If you want to return to your own implementation, remove the solution option from the build command.

For example, after using:

```
make CAMERA=solution OBJPARSER=solution
```

return to your own camera implementation with:

```
make clean
make
```

The normal make command uses the source code in your project rather than the camera solution library.

2.6 Multiple Solution Libraries

You can use more than one solution library at the same time.

For example:

```
make clean
make TRANSFORMS=solution GEOMETRY=solution
```

This tells the build system to use the supplied implementations for both transformations and geometry while continuing to compile all other topics from your source code.

Solution libraries can therefore be mixed with your own implementations. You do not have to replace the entire editor with a solution version.

For example:

```
make clean
make CAMERA=solution
```

means:

Camera	→	supplied solution library
Everything else	→	your source code

Similarly:

```
make clean
make TRANSFORMS=solution CAMERA=solution
```

means:

Transformations	→	supplied solution library
Camera	→	supplied solution library
Everything else	→	your source code

2.7 Important: Clean Before Switching Implementations

Always clean the project before changing a topic between your implementation and a solution library:

```
make clean
```

For example, if you first build with:

```
make CAMERA=solution
```

and later want to use your own implementation, do:

```
make clean
make
```

Likewise, if you are using your own implementation and want to switch to the solution:

```
make clean
make CAMERA=solution
```

Cleaning is important because previously compiled object files can remain in the project and interfere with the new build configuration.

2.8 Building a Full Solution Reference Editor

If you have been provided with all available solution libraries, you may build a fully functional version of the editor to use as a **reference while completing your assignments**.

This reference build can be useful when you want to compare your implementation against the completed behavior of the editor. For example, you can use it to check how a shape should render, how a control should respond, or how a later feature is expected to behave.

First, make sure all supplied `.a` files are located in:

```
lib/solution/
```

Then clean the project:

```
make clean
```

Build the editor using all available solution libraries:

```
make \
  TRANSFORMS=solution \
  GEOMETRY=solution \
  OBJPARSER=solution \
  CAMERA=solution \
  VRCAMERA=solution \
  CURVES=solution \
  SKINNING=solution \
  ANIMATION=solution \
  TEXTURES=solution \
  LIGHTING=solution \
  RAYTRACER=solution \
  BVH=solution \
  PARTICLES=solution
```

Then run:

```
./editor
```

This creates a version of the editor that uses the supplied solution implementation for every supported topic. The build system currently recognizes 13 solution topics.

You may use this full solution build as a **visual and behavioral reference** while developing your own code. For example, if you are unsure how your Cube, Sphere, transformations, camera controls, or another feature should behave, you can run the reference build and compare it against your own implementation.

The reference build does **not** replace the work required for the assignment. You are still responsible for implementing and submitting the source code required for your current assignment. Do not submit the solution libraries as your own implementation.

Also make sure that all solution libraries match your development environment. The `.a` files are specific to the operating system, CPU architecture, and compiler/toolchain being used.

2.9 Solution Libraries Do Not Replace Your Source Code

Using a solution library does **not** mean that you should copy someone else's source code into your project. The library is compiled code that is linked into your executable during the build process. Your source files remain in your project and can continue to be modified normally.

For example, when you build with:

```
make TRANSFORMS=solution GEOMETRY=solution
```

you can still edit and test your Assignment 3 files such as:

```
CameraControl.cpp
ObjParser.cpp
```

The previous assignment's functionality is simply being supplied by the library while you work on the rest of the assignment.

2.10 Summary

The most common commands you will use are:

```
# Normal build
make

# Clean compiled files
make clean

# Clean and rebuild
make clean && make

# Build using the camera solution
make clean
make CAMERA=solution

# Build using multiple solutions
make clean
make TRANSFORMS=solution GEOMETRY=solution
```

Remember: Solution libraries allow you to use completed functionality without replacing your own source code. They are provided so that you can continue making progress when a previous or future assignment component is not yet implemented.

3. Assignment Overview

3.1 Functionality

Assignment 3 consists of two implementation tasks that are intended to work together. The first task is the camera-control system. The camera uses an orbit model in which its position is derived from a target point and three spherical-coordinate parameters. The second task is the OBJ parser, which reads an external mesh from an input stream and converts the file's vertex, normal, and face records into the mesh representation already used by the editor.

Mesh loading – Some objects in the **Insert** menu are made with formulas. Real models are not. A scanned bunny, a sculpted character, anything an artist made in Blender or Maya arrives as a **file**, a long list of points and triangles with no formula behind it. You will write the parser that reads one.

Camera control - The first thing anyone does with a model they just loaded is try to look at it from another angle. In Assignment 2 you could not: the camera was nailed to **+Z**, and composing a scene meant moving the shapes instead. That was on purpose, and this is where you get the camera back, by writing it.

You may have noticed in Assignment 2 that dragging the mouse in the viewport did nothing at all. The functions it calls are empty. They are yours now. Once both components are complete, the editor should allow you to import an external mesh and then navigate around that mesh using the mouse. The camera controls are:

Left-Click:	Allows you to pan the camera up, down, left and right
Right-Click:	Allows you to rotate the camera
Mouse Wheel:	Allow you to zoom the camera in and out

The provided code already contains the UI, mouse-event routing, camera state, matrix setup, file dialog, mesh upload/drawing, and parseCorner() helper. Your job is to complete the missing behavior.

3.2 Defined Scope

The files you will be working on, and the section in those files that you will be working on are as follows:

Source File	Method to be Completed
`src/ObjParser.cpp`	`parseObj(std::istream&, ObjMesh&)`
`src/CameraControl.cpp`	`Camera::orbit()`
	`Camera::pan()`
	`Camera::zoom()`

Both compiles as given and both do nothing useful: importing a file produces an empty mesh, and the mouse does not move the camera. Those are your two starting points.

Some of the important aspects of the code that is important to review and understand before you begin any work:

Source File	Method to be Completed
The file dialog	**FileImporter** opens the chooser, hands you a stream, and registers the finished shape with the **ShapeManager**. The menu path to import your shape is as follows: **File > Import > Import Shape**
ImportShape	Uploads the mesh to the GPU and submits it for rendering. You do not need to redesign this portion of the pipeline.
parseCorner()	This is already implemented in **ObjParser.cpp**. It separates a face corner such as **a**/**b**/**c** into the appropriate zero-based indices and handles negative OBJ indices.
The camera's *state*	**Camera** holds **theta**, **phi**, **radius** and **target**, and **updatePosition()** turns them into an eye point. That function, **setView()**, the view and projection matrices and **generateRay()** are all given and all correct.
The mouse plumbing	**Renderer.cpp** already routes left-drag, right-drag and the scroll wheel to the three functions below, and already ignores drags that start on a panel.

4 Part 1 - Camera Control

4.1 The Model

The editor uses an orbit camera. Rather than storing the camera position directly, the camera maintains a target point and describes its position relative to that target using spherical coordinates. The three parameters are **radius (r)**, **theta (θ)**, and **phi (ϕ)**.

In an orbit camera system, you can visualize the components as follows:

target	The point in which the camera points to. The beginning target at the origin (0, 0, 0).
*radius (**r**)*	The distance from that center target out to the camera's location.
theta (θ)	Rotates the camera horizontally along the equatorial plane.
phi (ϕ)	Tilts the camera vertically down from the upper pole.

By changing θ and ϕ, the camera glides smoothly across the surface of an invisible sphere, always keeping its lens pointed inward toward the center target. Remember that θ and ϕ are depicted in *radians*.

`updatePosition()`	*given*	This is the one line that turns those values into an camera point

The functions to manage the camera transformation controls are as follows:

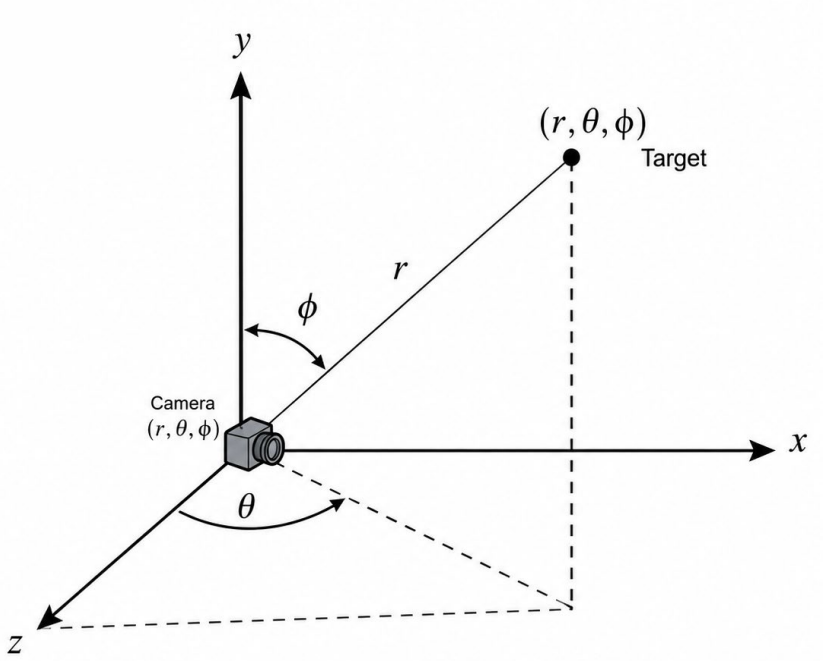
```
position.x = target.x + radius · cos(phi) · cos(theta);
position.y = target.y + radius · sin(phi);
position.z = target.z + radius · cos(phi) · sin(theta);
```

Your three functions change that state and then call `updatePosition()`. They are responsible for modifying the camera state and then calling `updatePosition()`. Changing the angles or radius without updating the position will change the stored state but will not move the camera.

That last step is not optional and it is the most common thing to forget: the angles change, and the camera does not move.

4.2 orbit($\Delta\theta$, $\Delta\phi$) - Right Drag

The target is to add the deltas to **theta** and **phi**, clamp the pitch, and then call `updatePosition()`.



where:

$\boldsymbol{r}$ = distance from camera to target
$\boldsymbol{\theta}$ = azimuth (rotation around y axis)
$\boldsymbol{\phi}$ = elevation (angle above or below horizontal)

The caller has already worked out the sign and scale of the deltas before they reach you. Add what you are given.

Clamp **phi** so the camera never reaches either pole. A suitable limit is $\pm(\frac{\pi}{2} - 0.1)$. Straight up is $\phi = +\pi/2$.

There, the direction the camera is looking becomes parallel to the up vector **(0, 1, 0)**, and `glm::lookAt` builds its basis by crossing those two. Because the cross product of parallel vectors is zero, normalizing zero is **NaN**, and the whole screen goes black or white and stays that way with nothing on it to control the camera.

This creates a degenerate camera basis and can result in invalid values and a blank viewport. A small margin from each pole is sufficient; the assignment uses 0.1 radians. The key is to stop a little short of both poles.

The calculation

```
glm::half_pi<float>() - 0.1f;
```

is plenty.

Do not clamp theta. Yaw is periodic. Horizontal rotation is periodic, so the camera should be able to continue rotating around the target indefinitely. cos/sin handle a $\boldsymbol{\theta}$ of **400 radians** perfectly well. A camera that jams after two turns is broken in a way nobody thinks to test.

Why clamp phi?
At the poles, the viewing direction becomes parallel to the world up vector (0, 1, 0). `glm::lookAt()` then cannot construct a valid camera basis from those vectors. The resulting normalization can produce **NaN** values and a black or otherwise invalid viewport.

4.3 pan(dxScreen, dyScreen) - Left Drag

The `pan()` method is called when the user performs a left-button drag. Panning does not change the camera's radius or orbit angles. Instead, it moves the target point, after which `updatePosition()` moves the camera by the same relative amount.

Panning moves what the camera is looking **at**. The eye follows for free, because `updatePosition()` derives it from the target.

The deltas arrive in screen space, x across the window, y down it, so they mean nothing until they are turned into world-space directions. The input values are expressed in screen coordinates, so they must be converted into directions in world space. Begin by determining the camera's forward and right directions. The camera's own basis is what does that:

```
forward = target – position;
right   = normalize(cross(forward, up));
```

Use the camera's right and up directions to move the target according to the supplied screen-space deltas. Pay particular attention to the sign of each movement. The intended behavior is that dragging the scene to the right moves the target in the corresponding opposite world-space direction so that the scene follows the mouse naturally.

Move **target** along **right** and along **up** by the two deltas, then call

```
updatePosition();
```

Two things to get right:

- **The signs** - Dragging right should carry the scene right, which means the target moves left. Get one backwards and the camera fights the mouse; get both backwards and it feels fine until you try to orbit.
- **Guard the degenerate case** - If the eye ever coincides with the target, forward is the zero vector and normalizing the cross product is **NaN** again. Bail out early when its length is tiny.

4.4 zoom(amount) - Scroll Wheel

This is a **dolly**, not a zoom: the field of view is untouched and the eye actually moves. Scrolling forward gives a **positive** amount and should bring the camera **closer**, so the radius goes down as the amount goes up. The `zoom()` method is called when the mouse wheel is moved. In this assignment, the operation is implemented as a dolly rather than a change to the camera's field of view. Increasing the forward scroll amount should move the camera closer to the target by reducing the radius.

Then call

```
updatePosition();
```

The radius must remain above a minimum value. Use a lower bound of 0.5, as assumed by the rest of the assignment. Without a lower bound, the camera can reach the target, producing an undefined viewing direction, and can eventually pass through the target and reverse the behavior of the controls.

```
radius = max(0.5, radius - amount);
```

Clamp the radius to a floor - 0.5 is what the rest of the class assumes. Without it, scrolling far enough puts the eye exactly on the target (no direction, **NaN** basis again), and then straight through it, at which point the scene is inside out and every other control works backwards.

4.5 The Two Traps, Together - Camera Testing and Common Errors

Both halves of this assignment can produce a black screen, and they look identical from the outside. If the viewport goes black and stays black after you drag, you have almost certainly hit a **NaN**, not a rendering bug.

Test each camera operation independently before combining them. Verify that horizontal and vertical orbiting behave continuously, that the camera can move around the target without becoming stuck, that panning moves the scene consistently with the mouse, and that repeated scrolling cannot reduce the radius below the required minimum.

A black or otherwise corrupted viewport immediately after camera movement is often caused by an invalid vector produced by a degenerate camera configuration. If this occurs, first verify the **phi** clamp, the **radius** floor, and the zero-length forward-vector check before investigating the rendering code.

Printing

```
camera.getPhi();
```

and

```
camera.getRadius();
```

will tell you in ten seconds.

5 Part 2 – The OBJ Parser

This is a large portion of the project, and one that you would need to pay close attention to. Once you've completed the previous task, we can move on to the tough part: loading new objects. In the sample code, we have provided several 3D meshes in Wavefront OBJ format. It is your job to write the code to load and display these files. Wavefront OBJ files are a fairly standard format that can describe all sorts of shapes, and you'll be handling a subset of their functionality.

Let's look at `sphere.obj`. It's a big file, but it can be summarized as follows:

```
#This file uses ...
...
v 0.148778 -0.987688 -0.048341
v 0.126558 -0.987688 -0.091950
...
vt 0.000000 0.050000
vt 0.050000 0.050000
...
vn 0.252280 -0.951063 -0.178420
vn 0.295068 -0.951063 -0.091728
...
f 22/23/1 21/22/2 2/2/3
f 1/1/4 2/2/3 21/22/2
...
```

The provided sample files use a limited subset of OBJ: vertices, normals, texture coordinates, and triangular faces. Your parser should read those records, keep the mesh data needed by the editor, and ignore unsupported records safely.

Each line of this file starts with a token followed by some arguments. The lines that start with **v** define vertices, the lines that start with **vn** define normals, the line that starts with **vt** define texture mapping coordinates (which you will use in Assignment 5), and the lines that start with **f** define faces. There are other types of lines, and your code should ignore these.

Your first task is to read in all of the necessary data into the correct data structure. Review the header and source files for the Shape and **ImportShape** classes. **ImportShape** is a child of Shape, so it inherits the common attributes

and methods that you worked with in your previous assignment. In the `ObjParser.h` header file, you will see the struct **ObjMesh** defined containing several defined vectors to store your vertices, normals, texture coordinates, and faces. You will populate a variable of type **ObjMesh** called out, and this will be returned to **ImportShape** to create your imported shape in the editor.

5.1 OBJ Records to Support

Reading the OBJ, you can reference the data type using the following chart:

Record	Type	Line in OBJ file sample	How to read data
v	vertex	`v 0.148778 -0.987688 -0.048341`	Read three floating-point values and add a vertex.
vn	normal	`vn 0.252280 -0.951063 -0.178420`	Read three floating-point values and add a normal.
vt	texture coords	`vt 0.375000 0.812500`	Read u and v texture coordinates and store them in a local lookup table. Ignore an optional third component if present.
f	faces	`f 22/23/1 21/22/2 2/2/3`	Read the first three face corners. Store vertex and normal indices for the triangle, and store one resolved UV coordinate per corner when available.
	others	`# comment, mtllib, usemtl, o, g, s`	Ignore safely.

Your `ObjMesh()` struct has vectors that will allow you to store the data from your imported OBJ file to the associated vector. The vertices (**v**) will go into the **vertices** vector as type `glm::vec3`, the **normals** (**n**) will go into the **normals** vector as type `glm::vec3`, and your texture coordinates (**vt**) will go into the **textCoord** vector as type `glm::vec2`. When reading a **vt** line in the OBJ file, if a third coordinate is present, you can disregard.

5.2 Face Corners

A face corner commonly has the form vertexIndex / textureIndex / normalIndex. The vertex index identifies a position, the texture index identifies a UV coordinate, and the normal index identifies a normal. Texture mapping will be used in a future assignment, but this parser should still resolve and store the UV coordinate now when the OBJ file provides one.

A triangular face has three corners. For example:

```
f 22/23/1 21/22/2 2/2/3
```

This creates a triangle using vertices 22, 21, and 2; texture coordinates 23, 22, and 2; and normals 1, 2, and 3. Store the face as vertex-normal index pairs in the mesh structure supplied by the starter code, and append one UV coordinate per corner to `texCoords` when valid texture indices are present.

OBJ indices are one-based and may also be negative, meaning they count backward from the end of the corresponding table. The supplied `parseCorner()` helper handles the supported corner forms and returns resolved zero-based indices. Use that behavior directly rather than subtracting a second time.

In the `ObjMesh()` struct, **vertices** is defined as an STL vector of `glm::vec3`. An STL vector is simply a list of arbitrary objects. In this case, it is a list of `glm::vec3` objects.

```
std::vector< glm::vec3> vertices;
```

To add a new entry to this array, use the built in method `push_back(glm::vec3)` to add a net tuple of vertices to the vector.

```
out.vertices.push_back(glm::vec3(x, y, z))
```

There are several ways to iterate over an STL vector. Here's an example of using indices (if you're interested in learning more about STL, check out the documentation at https://www.boost.org/sgi/stl/).

The editor is already using **FileImporter** class to import your OBJ file to into an <istream> data stream and passing it to your `parseObj()` method. You will pull each line from this stream into a string and parse it out to obtain the data for each line, then separate the line to check whether the line contains data that you need or if it should be disregarded:

```
while (std::getline(in, <string>line)) {
// ... your parsing code here
}
```

The dialog box will load the file's content into the standard input stream `std::istream`. This stream will be passed into `parseObj()` through the parameter in, and can be accessed using the `std::getline()` method. For example, to read a single line of data from the stream (all characters up to the next newline):

```
std::getline(in, line)
```

`std::getline()` will return zero at the end of the file. You can use this fact to step through each line in the file. Once you get have an array of characters (the text from a single line of the file), you can parse it using an **istringstream** object. Create a **istringstream** object from an array of characters (buffer) as follows:

```
std::istringstream ss(line);
```

Now that you have a **stringstream** object, you can read tokens (separated by spaces) from the buffer in order by using the **>>** operator. For example, given the input string "**`v 1.0 2.0 -2.0`**", in the following code:

```
glm::vec3 v;
string s;
ss >> s;
ss >> v[0] >> v[1] >> v[2];
```

will put the value **v** into **s**, and load the values **1.0**, **2.0**, and -**2.0** into **v[0]**, **v[1]**, and **v[2]**. Note that you can compare the string objects to constant strings using the regular **==** operator.

```
if (s == "v") {
  //do something
}
```

Make sure that you're able to load and view the eight provided files without crashing; these are the eight files we'll test your program on.

You may want to compile and run the provided sample solution using **make CAMERA=solution OBJPARSER=solution** and then running **./editor** to get an idea of how your application should work (use the UI interface to demo the functionality).

5.3 Parsing Strategy

In order to complete this part of the assignment, you can follow the following strategy to pull the data from the stream and save it to the proper data structure.

- Read the input stream one line at a time.
- Skip blank lines and comments.
- Use a **stringstream** to read the first token on each line.
- For **v** records, read x, y, and z and add a vertex.
- For **vn** records, read x, y, and z and add a normal.
- For **vt** records, read u and v into a local texture-coordinate table. If a third value appears on the line, ignore it.
- For **f** records, collect the face fields and process the first three corners as one triangle.
- Use `parseCorner()` to resolve vertex, texture, and normal indices, including optional and negative indices.
- Reject a face if any required vertex index is invalid.
- If a normal index is missing or invalid, fall back to the vertex index so the rest of the project can still handle the mesh consistently.
- Push one UV per triangle corner when a valid texture index is present, and set **hasTexCoords** only if the file actually supplied usable UV data.
- Ignore records that are outside the required subset.

5.4 Drawing and Persistence

Do not draw the mesh directly while parsing. OpenGL redraws the scene whenever the window is refreshed, resized, uncovered, or otherwise invalidated. The parser should store mesh data so the renderer can draw it repeatedly.
Texture coordinates should be stored for later use, but they should not change untextured rendering in this assignment. If the OBJ file contains no usable **vt** records, clear **texCoords** and leave **hasTexCoords** false so the shape keeps its untextured rendering path.

Your final executable should import the provided OBJ files through the editor's **Import Shape** file dialog. Sample OBJ files are located in the project's data folder.

5.5 Parser Failure Cases

A robust parser should not crash on lines it does not understand. It should ignore unsupported tokens and continue. For this assignment, you do not need to load materials, apply textures, or compute missing normals. If a face contains more than three corners, use the first three corners only; full polygon triangulation belongs elsewhere in the renderer pipeline.

6 Testing Your Assignment

Before submitting your assignment, test each required component independently.

6.1 Test Camera Controls

Place a shape in the center of your scene. From there

- Orbit horizontally in both directions and verify that the camera can complete multiple rotations around the target.
- Orbit vertically in both directions and verify that the camera stops before reaching the poles.
- Pan horizontally and vertically and verify that the target and camera move together.
- Scroll forward and backward repeatedly and verify that the radius never becomes less than 0.5.
- Combine orbiting, panning, and zooming and verify that the controls remain stable after repeated use.

6.2 Test OBJ Importing

Verify that each required shape can be inserted successfully:

- Import each of the four supplied OBJ files through File → Import → Import Shape.
- Verify that the complete model is displayed and that the application does not crash.
- Inspect the model from multiple viewpoints to identify missing triangles or incorrectly constructed faces.
- Check that the supplied vertex normals are associated with the appropriate face corners.

Download a shape from a reputable source. Some suggested sources include Sketchfab (https://sketchfab.com/tags/obj), Free3D (https://free3d.com/3d-models/obj), and CGTrader (https://www.cgtrader.com/free-3d-models). Try importing them and see if they function correct.

Note: Occasionally, you may run into an issue importing a downloaded obj 3D model. Before you suspect an issue with your code, test it with your solution binary and see if its an issue with the OBJ file. Some OBJ files may be incompatible depending on how you handle additional lines that is not required.

6.3 Test the Complete Feature Set

After testing the two components independently, import an OBJ model and use the camera controls to inspect it from several directions and distances. This final test is important because it exercises the two portions of the

assignment together and can reveal problems that are not apparent when each component is tested in isolation.

7 Create a Scene and Take a Screenshot

Once all required functionality is complete, create a scene demonstrating your work.

Your scene must contain at least one instance of each required Assignment 2 shape:

Once the required functionality is complete, create a scene that demonstrates the features implemented in Assignment 3. The scene should contain at least one imported OBJ model included in your library, and one that you have downloaded from a 3rd part source. Create your scene with a mixture of existing geometry nd imported OBJ models. Your models should be arranged so that the geometry is clearly visible in the editor viewport.

Use the camera controls to compose the scene rather than relying only on object transformations. Your final view should demonstrate that the camera can be positioned at a useful distance and angle and that the imported models can be inspected from a meaningful viewpoint.

Take a screenshot showing the complete editor window, including the viewport and the imported model. The screenshot should be clear enough to demonstrate that the OBJ importer produced visible geometry and that the camera has been moved from its initial viewpoint.

Arrange the objects so that all four are clearly visible. Make an awe-inspiring scene and your screenshot could make it to the CP 484 Hall of Fame.

8 Submission

As a final step, create a README.txt or README.md file containing the following information:

- Name of student
- Operating system/development environment used. Include version of OS and architecture type, as this helps me with compilation issues.
- Special instructions on how to compile and run your code if different from the instructions provided.
- Did you collaborate with anyone in the class? If so, describe who you worked with and what assistance was given or received.
- Were there any references, books, websites, or other resources that you found particularly useful? Please list them.
- Are there any known problems with your implementation? If so, describe them.
- Which OBJ files did you download and import into your code? Did you run into any issues? IF so what were they and how did you resolve.
- Did you implement any additional functionality?
- Do you have any comments about the assignment? Were any requirements unclear? Was any portion particularly difficult or useful?

Submit **one compressed archive** containing your Assignment 3 project.

Your submission should include:

- Your source code. You can include the entire codebase or just include the files that have been changed. (i.e. ObjParser.cpp, CameraControl.cpp etc.). If only providing the changed files, please provide instructions in your readme on how to integrate into the starting code to properly compile and run.
- Your compiled binaries of the required Assignment 3 functionality.
- Your README.txt or README.md.

- The external OBJ files you've downloaded to test and screenshot your scene.
- Your final scene screenshot of your masterpiece. It may be showcased in class.
- Any files required to compile and run your editor that are part of your project.
- Put all files in a zip or tar.gz file.
- You should only submit one compressed file with all required files. If your submission does not include one or more of these items, you will lose substantial points.

Before creating the final archive, verify that the project builds successfully from a clean state, that all four supplied OBJ files can be imported, that orbit, pan, and zoom behave correctly, and that the required camera safeguards are functioning.

Do Not Submit Solution Libraries as Your Work

You are responsible for submitting **your own Assignment 3 implementation**.

Using a provided solution library for functionality from another assignment, such as:

```
libcamera.a
```

does not replace any Assignment 3 requirement.

For example, using:

```
make CAMERA=solution
```

is permitted while completing Assignment 3, but it shouldn't replace your code as a solution. You must still submit your own Assignment 3 camera and OBJ parser code.